electronics

MDPI

Article

A Low-Latency Streaming On-Device Automatic Speech Recognition System Using a CNN Acoustic Model on FPGA and a Language Model on Smartphone

Jaehyun Park, Hyeonkyu Noh, Hyunjoon Nam, Won-Cheol Lee and Hong-June Park *

Department of Electrical Engineering, Pohang University of Science and Technology (POSTECH), Pohang 37673, Korea; pjh0905@postech.ac.kr (J.P.); nohhg92@postech.ac.kr (H.N.); namhj@postech.ac.kr (H.N.); wclee1105@postech.ac.kr (W.-C.L.)
* Correspondence: hjpark@postech.ac.kr

Abstract: This paper presents a low-latency streaming on-device automatic speech recognition system for inference. It consists of a hardware acoustic model implemented in a field-programmable gate array, coupled with a software language model running on a smartphone. The smartphone works as the master of the automatic speech recognition system and runs a three-gram language model on the acoustic model output to increase accuracy. The smartphone calculates and sends the Mel-spectrogram of an audio stream with 80 ms unit input from the built-in microphone of the smartphone to the field-programmable gate array every 80 ms. After ~35 ms, the field-programmable gate array sends the calculated word-piece probability to the smartphone, which runs the language model and generates the text output on the smartphone display. The worst-case latency from the audio-stream start time to the text output time was measured as 125.5 ms. The real-time factor is 0.57. The hardware acoustic model is derived from a time-depth-separable convolutional neural network model by reducing the number of weights from 115 M to 9.3 M to decrease the number of multiply-and-accumulate operations by two orders of magnitude. Additionally, the unit input length is reduced from 1000 ms to 80 ms, and to minimize the latency, no future data are used. The hardware acoustic model uses an instruction-based architecture that supports any sequence of convolutional neural network, residual network, layer normalization, and rectified linear unit operations. For the LibriSpeech test-clean dataset, the word error rate of the hardware acoustic model was 13.2% and for the language model, it was 9.1%. These numbers were degraded by 3.4% and 3.2% from the original convolutional neural network software model due to the reduced number of weights and the lowering of the floating-point precision from 32 to 16 bit. The automatic speech recognition system has been demonstrated successfully in real application scenarios.

Keywords: convolutional neural network; end-to-end; streaming automatic speech recognition; real-time processing; latency

check for updates

Citation: Park, J.; Noh, H.; Nam, H.; Lee, W.-C.; Park, H.-J. A Low-Latency Streaming On-Device Automatic Speech Recognition System Using a CNN Acoustic Model on FPGA and a Language Model on Smartphone. *Electronics* **2022**, *11*, 1831. https://doi.org/10.3390/electronics11121831

Academic Editor: Nikolay Hinov

Received: 9 March 2022
Accepted: 3 June 2022
Published: 9 June 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Deep-learning technology has been successfully applied to automatic speech recognition (ASR) [1,2]. A powerful computer with graphics processing units (GPUs) is mainly used to train the ASR model to achieve a word error rate (WER) of less than a few percent using hundreds of millions of weights. Most of the high-accuracy ASR models are full-context models, which wait to hear the complete utterance before generating output [3–8]. On the contrary, streaming ASR models try to generate output as fast as possible without waiting for the completion of utterance [9,10]. An ASR model that is capable of streaming operation with reasonable WER is required for artificial intelligence speakers or on-device transcription applications. For streaming operation, the model-processing time should be shorter than the unit input time. The minimum latency of the state-of-the-art streaming

ASR model is reduced to 120 ms [9], which is larger than the latency specification of 10 ms in hearing aids but is smaller than the latency (150 to 230 ms) of the public switched telephone.

Existing streaming ASR models mostly use high-speed cloud computers, and therefore suffer from potential lack of privacy, and from communication delays to and from the computers. To avoid these problems, low-latency streaming on-device ASR models must be able to run ASR operations in a compact hardware system; this kind of compact hardware ASR system can be used to deploy the ASR capability in electronic home appliances.

Streaming on-device ASR systems have been successfully implemented using the CPU (application processor) in a smartphone; ref. [11] achieved WER = 7.3% with 117 M weights, and [12] achieved WER = 6.7% with 39 M weights. However, a microcontroller is preferred over a smartphone to implement a compact hardware ASR system for inexpensive electronic home appliances. Although a microcontroller performs rather simple speech-command recognition, it has limited hardware resources, so it takes too long to process a general ASR operation that has millions of weights. Instead of microcontrollers, general ASR can be implemented using an application-specific integrated circuit (ASIC) chip along with a DRAM chip and input and output devices. The ASR operation consists of two steps: an acoustic model (AM) and a language model (LM). An AM is easier than an LM to implement in an ASIC chip, because LM has a multi-gigabyte dictionary that requires a large memory. Additionally, an AM can perform the ASR operation without using an LM, but with some accuracy degradation.

Most of the recently published hardware AM for streaming ASR (Table 1) apply a uni-directional long-short-term-memory (LSTM) model with shallow layers [13–17] (2 or 3-layer LSTM, 7-layer CNN). An accurate low-latency hardware AM is proposed for streaming ASR; a multi-layer CNN rather than a shallow-layer LSTM is chosen to enhance accuracy, and the unit input length is reduced to the smallest (80 ms) and a direct DRAM interface rather than an indirect DRAM interface such as Advanced eXtensible Interface (AXI) is employed for low-latency streaming operation. A 55-layer CNN AM is used in this work with 9.3 M weights because increasing the number of layers increases accuracy in neural networks, while less than 1 M weights are used in the previous works with shallow LSTM AM.

Table 1. Comparison of recently published hardware AM for ASR.

HW AM	Model/# Layers	#Weights	HW Complexity (L, O) [1]	Unit Input Length (ms)	DRAM Interface	Implementation
[13]	uniLSTM/2	972 K	294 K, 4.2	1530	AXI	FPGA
[14]	uniLSTM/2	820 K	307 K, 4.1	390	-	FPGA
[15]	uniLSTM/2	820 K	289 K, 6.4	1530	AXI	FPGA
[16]	uniLSTM/3	384 K	-, 0.3	110	-	ASIC
[17]	CNN/7	20.3 K	780 K, 0.05	165	-	ASIC
This work	CNN/55	9.3 M	124 K, 0.27	80	Direct interface	FPGA

[1] L: #LUTs on FPGA, #NAND2 gates on ASIC, O: on-chip memory capacity (MB).

In this work, the AM is implemented in FPGA as a preliminary step toward a compact ASIC ASR system. The AM is based on a CNN model [18]. Compared to the base model, the unit input length is reduced from 1000 ms to 80 ms to reduce latency, the number of frequency bins is reduced from 80 to 32, and the number of weights is reduced from 115 M to 9.3 M; this helps to achieve the low-latency streaming operation by reducing the computation time. A two-dimensional array (16 × 16 systolic array) is used in FPGA to reduce the interconnect routing complexity, and the input and output numbers of the CNN model are unified to 16 to increase the hardware utilization efficiency. A DRAM controller is implemented in FPGA for a direct interface between DRAM and FPGA. The LM is implemented in a smartphone to increase accuracy.

The resultant CNN model of this work has an input receptive field that is larger than an average sentence length of 10 s as in [18]; each convolution layer pads the previous input and the future input to the left and right side of the current input, respectively, to expand the input receptive field to 10,990 ms (Table 2). Whereas [18] includes the future input ranging from 250 ms to 5000 ms to increase accuracy, no future input is included in this work because the future input adds directly to the latency. By including the previous input of 11,590 ms with the current input of 80 ms in this work, the input receptive field at the final layer is 11,670 ms, and the minimum latency is 80 ms (Table 2).

Table 2. Comparison of the base model [18] and a simplified model of this work.

	Past Field (ms)	Current Field (ms)	Future Field (ms)	Input Receptive Field (ms)	Minumum Latency (ms)
Base model [18]	9990	750	250–5000	10,990–15,740	1000–5750
This work	11,590	80	0	11,670	80

To demonstrate the AM, a low-latency streaming on-device ASR system is implemented by combining the FPGA chip and a DRAM chip with an Android smartphone (Figure 1). An ASR driver program running on the smartphone controls the ASR system as a master. The ASR driver program receives an audio input stream through a built-in microphone of the smartphone, converts the audio data to a Mel-spectrogram, and sends it to the DRAM chip through the FPGA chip every 80 ms. The FPGA chip processes the 80 ms Mel-spectrogram in ~35 ms, then sends the calculated probability set of all word-pieces (AM output) to the smartphone. The ASR driver program generates text output by applying a three-gram LM to the received word-piece probability and displays the text output on the smartphone. The resultant ASR system gives the measured WER = 13.2% for the hardware AM and WER = 9.1% for the software LM; they are degraded by 3.4% and 3.2% compared to the full-software base model [18]. It is estimated that the WER degradation of this work is caused by the reduction in the number of weights from 115 M to 9.3 M and reducing the floating-point precision from 32 bit to 16 bit.

Section 2 explains the low-latency streaming on-device CNN AM of this work. Section 3 shows the nine instructions used to implement the CNN AM. Section 4 describes the hardware architecture of the CNN model. Section 5 presents the implementation and experimental results. Section 6 discusses the results of this work. Section 7 concludes this work.

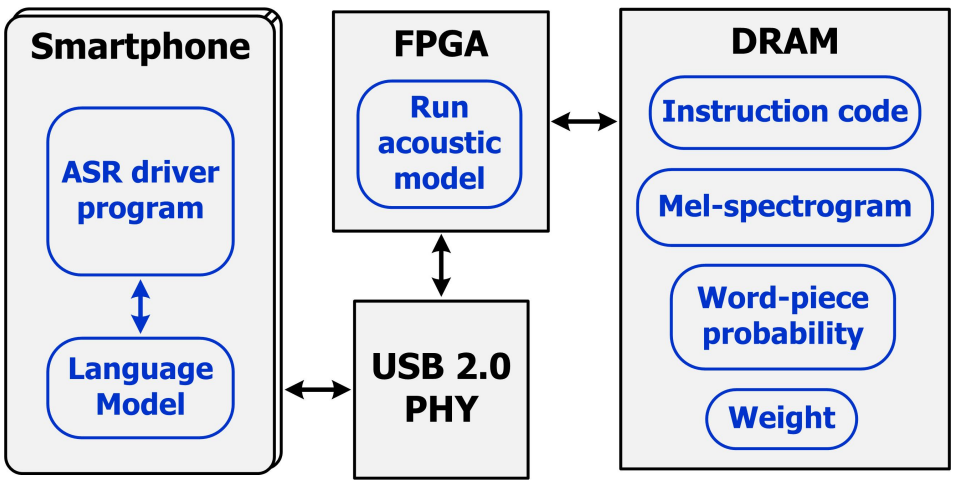


Figure 1. Proposed streaming ASR system.

2. Low-Latency Streaming On-Device CNN Acoustic Model

To implement a low-latency streaming on-device AM, a CNN acoustic model [18] was chosen as the base model and modified for low-latency hardware implementation. The latency of the AM is composed of the unit input time and the model processing time. The unit input time is the time interval of speech that the AM processes; it is 1000 ms in the base model [18], and is composed of the current 750 ms and the future 250 ms. In this work, to achieve low-latency operation, the unit input time is reduced to 80 ms (no future data) or 160 ms (80 ms current + 80 ms future); 80 ms and 160 ms were chosen because the time to utter an English syllable ranges from 60 ms to 150 ms [19]. To minimize the model processing time without losing much accuracy and to reduce hardware memory requirement, the number F of frequency bins in the Mel-spectrogram was reduced from 80 to 32, and the word-piece (AM output) was reduced from 10,000 to 648. The speech waveform is divided into time frames of 10 ms step with a 25 ms window; each time frame at 10 ms intervals is converted to a Mel-spectrogram.

The AM was trained using the wav2letter++ framework. Connectionist temporal classification (CTC) loss is used as a loss function to compare the alignment of time sequences between speech input and word-piece output. Training used the stochastic gradient descent (SGD) optimizer with an initial learning rate of 0.15. The spec augmentation is applied to randomize input data to prevent overfitting.

The AM of this work consists of cascaded two-dimensional convolution operation (CONV2D) and time-depth-separable (TDS) followed by the final fully connected (FC) layer. The TDS operation is composed of a series connection of a CONV2D and two FC layers with two residual networks [18] (Table 3). The AM accepts the Mel-spectrogram of 8 or 16 frames as input, and generates a set or two sets of probabilities for 648 word-pieces, respectively, whereas the base model [18] accepts the Mel-spectrogram of 100 frames as input and generates 12 sets of probabilities for 10,000 word-pieces. The number (N_o) of outputs of all layers is unified to 16 in this work except the final FC layer to improve utilization of the hardware computing unit. The number of weights is dominated by FC layers; i.e., two FC layers in a TDS operation and the final FC layer. An FC layer in a TDS operation has FxN_ixFxN_o weights, and the final FC layer has FxN_ix (number of word pieces) weights. To reduce the number of weights, F was reduced from 80 to 32 and N_o was reduced to 16 at the CONV2D and TDS layers and from 10,000 to 648 at the last FC layer.

Table 3. Comparison of CNN model between the base model [18] and this work (R—#repetitions; I—input; T_i—#input time frames (past + current + future); F_i—#input frequency bins; N_i—#inputs; O—output; T_o—#output time frames; F_o—#output frequency bins; N_o—#outputs; K—kernel shape of CONV2D; S—time stride). The unit input time is 80 ms in this work. Numbers inside parentheses for an input time frame refer to the past and the current and the future time frames, respectively.

	Base Model [18]					This Work				
Operation	**R**	**I: $T_i \times F_i \times N_i$**	**K**	**O: $T_o \times F_o \times N_o$**	**S**	**R**	**I: $T_i \times F_i \times N_i$**	**K**	**O: $T_o \times F_o \times N_o$**	**S**
CONV2D	1	(8 + 75 + 25) × 80 × 1	10 × 1	50 × 80 × 15	2	1	(8 + 8 + 0) × 32 × 1	10 × 1	4 × 32 × 16	2
TDS	2	(8 + 50 + 0) × 80 × 15	9 × 1	50 × 80 × 15	1	2	(8 + 4 + 0) × 32 × 16	9 × 1	4 × 32 × 16	1
CONV2D	1	(8 + 50 + 0) × 80 ×15	10 × 1	25 × 80 × 19	2	1	(8 + 4 + 0) × 32 × 16	10 × 1	2 × 32 × 16	2
TDS	3	(8 + 25 + 0) × 80 × 19	9 × 1	25 × 80 × 19	1	3	(8 + 2 + 0) × 32 × 16	9 × 1	2 × 32 × 16	1
CONV2D	1	(10 + 25 + 0) × 80 × 19	12 × 1	12 × 80 × 23	2	1	(10 + 2 + 0) × 32 × 16	12 × 1	1 × 32 × 16	2
TDS	4	(10 + 12 + 0) ×80 × 23	11 × 1	12 × 80 × 23	1	12	(10 + 1 + 0) × 32 × 16	11 × 1	1 × 32 × 16	1
CONV2D	1	(10 + 12 + 0) × 80 × 23	11 × 1	12 × 80 × 27	1	-	-	-	-	-
TDS	5	(10 + 12 + 0) × 80 × 27	11 × 1	12 × 80 × 27	1	-	-	-	-	-
FC	1	12 × 2160	-	12 × 10 K [1]	-	1	1 × 512	-	1 × 648 [1]	-

[1] #word-pieces.

Comparison with the base model in software shows that the number of weights is reduced by 12.4 times, whereas the computational complexity is reduced by 100 times and 50 times for the unit input times of 80 ms and 160 ms, respectively, in this work, with word error rate (WER) degradation less than 1% (Table 4). The 960 h LibriSpeech dataset [20] was used for training, and trained AM was tested for LibriSpeech test-clean dataset. WER is degraded by 0.1% in the 80 ms unit input (no future data) compared to the 160 ms unit input (80 ms future data).

Table 4. Comparison with the base model [18] in software (32-bit floating-point, 960 h LibriSpeech for training, LibriSpeech test-clean for test).

Model	**Base Model [18]**	**This Work**	
#weights	115 M	9.3 M	9.3 M
Unit input (ms)	1000	80	160
#input frequency bins	80	32	32
#output word-pieces	10,000	648	648
#layers	47	55	55
#MACs	1.64 G	16.4 M	32.7 M
AM WER (%)	9.8	10.7	10.6
LM WER (%)	5.9	6.7	6.6

3. Instruction Set for CNN Acoustic Model

To increase the programmability in the hardware implementation of the proposed AM, an instruction set with nine instructions was used in this work (Table 5); all the parameter values in Table 3 as well as the operation sequence of CONV2D, FC, residual network (RESNET), rectified linear unit (RELU), and layer normalization (LN) can be modified by changing the instruction code without changing the hardware.

The entire AM of this work (Table 3) is described with 334 lines of 64-bit instructions; they occupy 65% of the 4 kB instruction memory. To demonstrate the usage of the instructions, we illustrate the first TDS operation of Table 3 (input; (8 + 4 + 0) × 32 × 16, kernel shape; 9 × 1 each, #outputs; 16) (Figure 2) and the corresponding instructions (Figure 3). The first operation of the proposed AM is a CONV2D (Table 3); it accepts the Mel-spectrogram data with 32 frequency bins for 16 consecutive time-frames (160 ms: past 80 ms + current 80 ms) as input, applies 16 kernels of 10 × 1 shape each to the input with stride 2, generates 16 output data of four frames with 32 frequency bins each, and stores the output in buf-1. The first TDS operation concatenates the 32 × 16 data of the current four frames and the past eight frames generated by the first CONV2D operation and forms the 12-time frame data of 32 × 16 each. It applies 16 kernels of 9 × 1 shape each on the 12-frame data along the time-frame axis without mixing in the frequency axis, and generates 16 output data of four frames with 32 frequency bins. Then it applies two FC operations with a kernel of 512 × 512 along the frequency axis without mixing in the time-frame axis to generate 4 × 512 output data (Figure 2).

In the instruction code of the first TDS operation (Figure 3), a buffer is initialized for accumulation by the INI instruction before a CONV2D and two FC operations. Additionally, the layer normalization is performed after each residual network to improve training.

Table 5. Instruction set for hardware AM.

Instruction	Meaning	Parameters	
		Related to Input	Related to Output
LDIN	Load input from DRAM	#data(256b unit), buf-target, dram_in_addr	-
CONCAT_PAST	Concatenate previous and current input data	buf-past, buf-past_addr, past_data, buf-current, current_data	buf-concat
CONV2D/FC	Perform a 2D convolution (CONV2D) operation or a fully connected (FC) operation	type(0: CONV2D, 1: FC), buf-in, conv(fc)_in, kernel_shape, time_stride, dram_weight_addr	buf-out, conv(fc)_out
RESNET	Perform a residual network	buf-inA, buf-inA_addr, buf-inB, buf-inB_addr	buf-out, res_out
RELU	Perform a rectified linear unit	buf-in	buf-out
LN	Perform a layer normalization	buf-in, gamma, beta	buf-out
INI	Initialize a target buffer	buf-target, ini_data	-
STOUT	Store output to DRAM	#data(256b unit), buf-target, dram_out_addr	-
FINISH	Finish the acoustic model	-	-

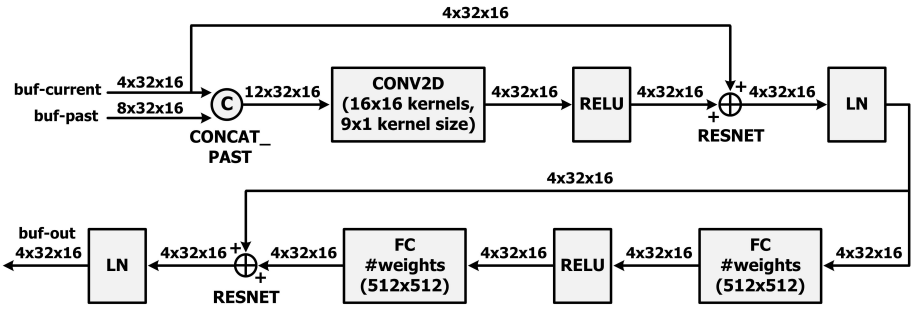


Figure 2. A single TDS operation of the first TDS block in Table 3.

	64bit
1	**CONCAT_PAST** buf-past*(buf-past)*, *buf-past addr*, past_data*(8x32x16)*, buf-current*(buf-1)*, current_data*(4x32x16)*, buf-concat*(buf-2)*
2	**INI** buf-target*(buf-1)*, ini_data*(4x32x16)*
3	**CONV2D** type*(0)*, buf-in*(buf-2)*, conv_in*(12x32x16)*, kernel_size*(9x1)*, time_stride*(1)*, *dram_weight_addr*, buf-out*(buf-1)*, conv_out*(4x32x16)*
4	**RELU** buf-in*(buf-1)*, buf-out*(buf-1)*
5	**RESNET** buf-inA*(buf-1)*, *buf-1_addr*, buf-inB*(buf-2)*, *buf-2 addr*, buf-out*(buf-1)*, res_out*(4x32x16)*
6	**LN** buf-in*(buf-1)*, gamma*(4.3)*, beta*(-0.9)*, buf-out*(buf-1)*
7	**INI** buf-target*(buf-2)*, ini_data*(12x32x16)*
8	**FC** type*(1)*, buf-in*(buf-1)*, fc_in*(4x32x16)*, kernel_size*(-)*, time_stride*(-)*, *dram_weight_addr*, buf-out*(buf-2)*, fc_out*(4x32x16)*
9	**RELU** buf-in*(buf-2)*, buf-out*(buf-3)*
10	**INI** buf-target*(buf-2)*, ini_data*(4x32x16)*
11	**FC** type*(1)*, buf-in*(buf-3)*, fc_in*(4x32x16)*, kernel_size*(-)*, time_stride*(-)*, *dram_weight_addr*, buf-out*(buf-2)*, fc_out*(4x32x16)*
12	**RESNET** buf-inA*(buf-1)*, *buf-1_addr*, buf-inB*(buf-2)*, *buf-2 addr*, buf-out*(buf-1)*, res_out*(4x32x16)*
13	**LN** buf-in*(buf-1)*, gamma*(1.6)*, beta*(0.2)*, buf-out*(buf-1)*

Figure 3. Instruction set for a single TDS operation of the first TDS block in Table 3.

4. Hardware Implementation of Proposed CNN Acoustic Model

The proposed low-latency streaming CNN AM was implemented in an FPGA for inference application. The FPGA inference chip (Figure 4) consists of four parts: AM manager, AM engine, DRAM controller, and USB 2.0 LINK.

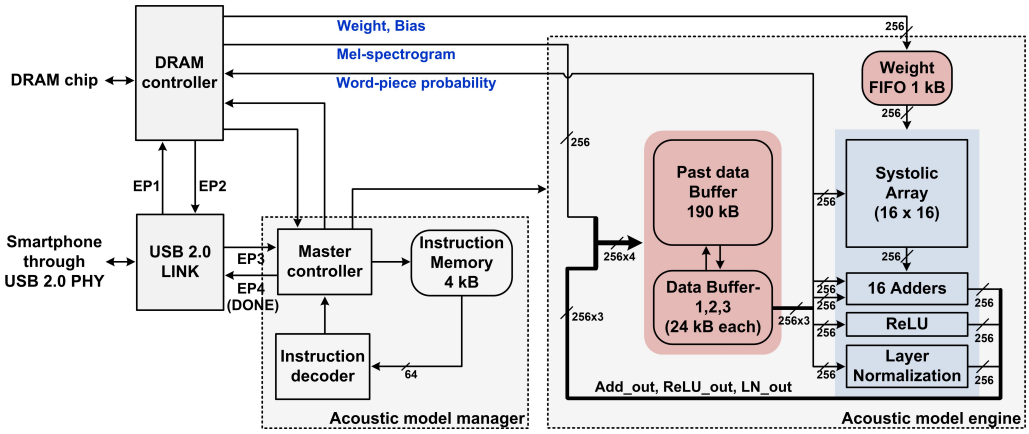


Figure 4. Hardware architecture of proposed AM.

The FPGA inference chip is connected to an Android smartphone through a commercial USB 2.0 PHY chip and to a DRAM chip, and is driven by the ASR driver program resident in the Android smartphone. The USB 2.0 LINK has four end-points (EP1, EP2, EP3, EP4) as well as the default end-point (EP0) for the bi-directional data interface between the smartphone and the FPGA chip. EP1 sends the instruction code, the weights, and the Mel-spectrogram of audio input to the DRAM chip through the dram controller. EP2 sends the 648 word-piece probability output from the DRAM chip to the smartphone through the dram controller. Through EP3, the ASR driver program sends three kinds of control signals to the master controller (MC) in the AM manager; one to fetch the instruction code from the DRAM chip to the instruction memory (INSTR_FETCH), another to run the instruction code after storing of each 80 ms audio Mel-spectrogram datum in the DRAM chip (START_RUN), and the other to reset all the registers in the AM manager and the AM engine (SOFT_RST). Through EP4, MC sends the DONE signal to the smartphone after storing each 648 word-piece probability in the DRAM chip.

When it receives the control signal to run the instruction code, the MC fetches a 64-bit instruction code from the instruction memory to the instruction decoder, then generates control signals for the AM engine and the DRAM controller by considering the instruction decoder output. The MC finite state machine (FSM) activates one of nine sub-FSMs (one for each instruction in Table 5) depending on the four most significant bits of the 64-bit instruction code. When the activated sub FSM is finished, the MC fetches the next instruction code to the instruction decoder. When the FINISH instruction is reached at the end of the instruction code, the MC FSM enters the IDLE state and stays there until it receives the control signal to run the instruction code through EP3 after 80 ms from the proceeding control signal. MC sends and receives the control and timing signals to and from the DRAM controller and the AM engine to fetch the 80 ms Mel-spectrogram data input, the weight, and the bias from the DRAM chip to the AM engine and to store the word-piece probability output from the AM engine in the DRAM chip.

The DRAM data rate is limited to 5 Gbps in this work, so a 512 × 16 bit FIFO is placed in the AM engine to prefetch weights and biases. Three data buffers (24 kB dual-port RAM each) and a past data buffer (190 kB single-port RAM) are used for input and output in the AM engine; this arrangement enables execution of combinations of convolution and residual network without DRAM access except the input loading of 80 ms Mel-spectrogram data and the output storage of the 648 word-piece probabilities, while the weight is prefetched from DRAM in the background. A 16-bit floating-point (FP16) format is used for all numbers, including the trained weights, bias, and the Mel-spectrogram input. The trained weights and bias in an IEEE 32-bit floating-point (FP32) format are converted into the FP16 format for inference.

The rest of this section describes the convolution and the fully connected operations in the AM engine. The FPGA inference chip performs the convolution and the fully connected operations in a pipelined fashion.

4.1. Processing Element

The calculating core of the AM engine is implemented with a square-shaped (M × M) systolic array with the same input and output counts to maximize the utilization efficiency of the systolic array across successive deep-learning layers; M was chosen to be 16 to fit within the FPGA capacity with 576 digital signal processing (DSP) units because a processing element (PE) consists of one DSP unit. The input and output counts of all the AM layers of this work are unified to 16 for efficient use of the systolic array. Each PE of the systolic array performs an FP16 multiply-and-accumulate (MAC) operation. The systolic array at column x and row y has three inputs $w_{x,y}$, $i_{x,y}$, $o_{x,y}$ and three outputs $w_{x,y+1}$, $i_{x,y+1}$, $o_{x+1,y}$ (Figure 5); it multiplies the stored weight $w_{x,y}$ by the bottom input $i_{x,y}$, adds the multiplier output to the left input $o_{x,y}$ and generates the output $o_{x+1,y}$. For weight reuse, the weight $w_{x,y}$ is stored and does not change, whereas the input $i_{x,y}$ changes at every system clock-cycle (120 MHz). Double buffering of the weight register is used to hide the weight update time. Due to the speed limit of the FPGA, the 16-bit multiplier and the 16-bit adder use three-stage and four-stage pipelines, respectively. The multiplier has a delay of three periods T of the 120-MHz system clock; to compensate for the delay, the left input $o_{x,y}$ is delayed by 3 T with respect to the bottom input $i_{x,y}$. The adder has a delay of 4 T, so the output $o_{x+1,y}$ is delayed by 4 T with respect to the left input $o_{x,y}$. Therefore, the bottom input $i_{x+1,y}$ of column $x + 1$ and row y is delayed by 4 T with respect to $i_{x,y}$. Additionally, the output $o_{x+1,y}$ is delayed by 7 T with respect to input $i_{x,y}$ due to the combined delay of the multiplier and the adder ($3T + 4T$) (Equation (1)).

$$o_{x+1,y}[n] = w_{x,y} \times i_{x,y}[n-7] + o_{x,y}[n-4], (o_{0,y} = 0) \quad (1)$$

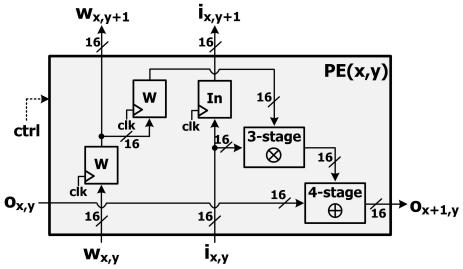


Figure 5. A processing element of systolic array.

Due to the bottom input shift register (Figure 5. "In") delay of 1 T, the output $o_{x,y+1}$ of row $y + 1$ at the same column is delayed by 1 T with respect to $o_{x,y}$. To get the time-synchronized output, the output $o_{15,y}$ of the last column is delayed by $(15–y)T$ with the output shift registers for $y = 0, 1, 2, \ldots, 15$. To maintain the 4 T time delay between $i_{x,y}$ and $i_{x+1,y}$ of the systolic array at adjacent columns, 4× × shift registers are placed at column x for $x = 0, 1, 2, \ldots, 15$ between the original 16-bit input and the bottom input $i_{x,0}$ of the systolic array. The entire systolic array block, a combination of the 16 × 16 systolic array and the input and output triangular-shaped shift registers, takes 16 FP16 numbers as input and generates 16 FP16 numbers as output in 82 T; the latency of 82 T consists of the input shift register delay 60 T, the row propagation delay 15 T of the systolic array, and the MAC delay 7 T of the PE at the 15th row and the 15th column.

4.2. Operation for Convolution Model

To explain the operation of the systolic array for convolution, the CONV2D layer shown in Figure 2 is taken as an example (Table 6). The CONV2D operation takes input data of 12 × 32 × 16 FP16 numbers (IN) from the second buffer (buf-2) and stores an output data of 4 × 32 × 16 FP16 numbers (OUT) to the first buffer (buf-1) by using 16 × 16 CNN kernels of 9 × 1 (KERNEL) elements in each kernel; both the input n_i and the output n_o of the CONV2D operation are 16. Each of the three buffers (buf-1, buf-2, buf-3) has 768 addresses, with each address corresponding to 16 FP16 numbers. The input buffer (buf-2) is addressed by t_i+12f and the output buffer (buf-1) is addressed by t_o + 4f; the frequency bin f is preserved because of the one-dimensional kernel shape (9 × 1).

Table 6. Parameters of an example CONV2D operation in Figure 2.

Opcode/Type	Buf-In	In_s hape	Kernel_shape	Stride	DRAM_w_addr	Buf-out	Out_shape
CONV2D	buf-2	12(t_i) × 32(f) × 16(n_i)	9 × 1	1	DRAM address of this CONV2D weights	buf-1	4(t_o) × 32(f) × 16(n_o)

The output of the CONV2D operation at the time frame t_o, the frequency bin f, and the output n_o (OUT[t_o,f,n_o]) is expressed by the convolution of KERNEL and IN (Equation (2)). To fit this computation on the 16 × 16 systolic array, a partial sum ($PSUM_{CONV}$) is calculated for a fixed set of k, t_o, and f; the MAC operation of 16 input values (n_i) is performed in parallel by the input parallelism of the systolic array, and the 16 output values (n_o) are calculated simultaneously by the output parallelism of the systolic array (Equation (3)). The $PSUM_{CONV}$ for different k values (0, 1, 2, …, 8) is accumulated using 16-parallel FP16 adders (Equation (4)).

$$OUT[t_o, f, n_o] = BIAS[n_o] + \sum_{n_i=0}^{15} \sum_{k=0}^{8} KERNEL[k, n_i, n_o] \times IN[t_o + k, f, n_i] \quad (2)$$

$$PSUM_{CONV}[k, t_o, f, n_o] = \sum_{n_i=0}^{15} KERNEL[k, n_i, n_o] \times IN[t_o + k, f, n_i] \quad (3)$$

$$OUT[t_o, f, n_o] = BIAS[n_o] + \sum_{k=0}^{8} PSUM_{CONV}[k, t_o, f, n_o] \quad (4)$$

For the first (k = 0) calculation of $PSUM_{CONV}$, the 256 weights of KERNEL[0,n_i,n_o] are loaded into the systolic array from DRAM through the FIFO in 129 T, which is the sum of the DRAM fetch time 108 T and the systolic array load time 21 T. The DRAM fetch time (108 T) includes the DRAM data request time (4 T), the DRAM-to-FIFO fetch time (98 T), the weight arrangement time (3 T), and the FIFO delay (3 T); the effective DRAM bandwidth is 5 Gbps and the system clock frequency is 120 MHz. $PSUM_{CONV}$[k,t_o,f,n_o] is computed by the systolic array for all k = 0, 1, 2, . . . , 8 (Equation (3)), and they are accumulated at the addresses t_o+4f of the output buffer (Equation (4)) as INT_PSUM_{CONV}[8,t_o,f,n_o] (Figure 6); the entire procedure for nine k values takes (108 + 21 + 7 + 128 × 9 + 82 + 4 + 3) T = 1377 T (Table 7). Because of the pipeline architecture, the longest delay (128 T) dominates the processing time for each k value and is added nine times; other delay components are added only once to the processing time for nine k values; the result corresponds to a throughput of 25.7 GMAC/s (30.7 Peak) at the system clock of 120 MHz. The output buffer (buf-1) is used as the input buffer of the subsequent operation.

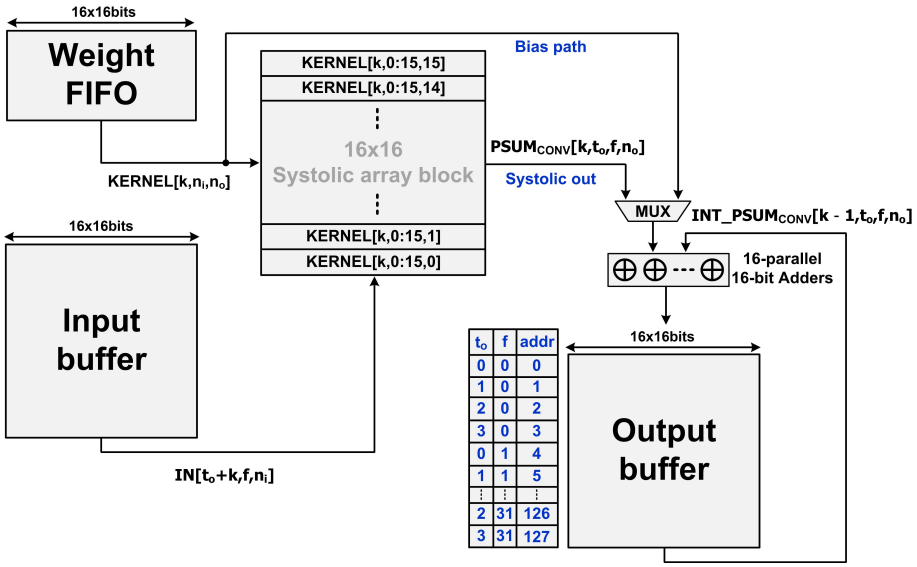


Figure 6. Hardware operation of CONV2D example in Table 6.

Table 7. Clock cycles for each pipeline stage in an example CONV2D operation of Figure 2.

Stage	DRAM Fetch	Weight Load	Input Fetch	Input Cycles	Systolic Array MAC	16-Adders Sum	Result Store
Cycle	108	21	7	128	82	4	3

4.3. Operation for Fully Connected Model

To explain the operation of the systolic array for the FC operation, the first FC layer of Figure 2 is taken as an example (Table 8). The FC layer has 512 × 512 weights, accepts a flattened input of 512 elements and generates a flattened output of 512 elements (Equation (5)) for a time frame t (20 ms duration in this example); an element is an FP16 number.

$$OUT[f_o \times n_o, t] = BIAS[f_o \times n_o] + W[f_o \times n_o, f_i \times n_i] \times IN[f_i \times n_i, t] \quad (5)$$

Table 8. Parameters of an example FC operation in Figure 2.

Opcode/Type	Buf-in	In_shape	#Weights	DRAM_w_addr	Buf-out	Out_shape
FC	buf-1	4(t) × 512(f_i × n_i)	512(f_i × n_i) × 512(f_o × n_o)	DRAM address of this FC weights	buf-2	4(t) × 512(f_o × n_o)

The 512 input elements consist of f_i×n_i (32 × 16) elements; n_i (16) elements of f_i are stored at the address of t+4f_i in the input buffer (buf-1). The output 512 elements consist of f_o × n_o = 32 × 16 elements; n_o = 16 elements of a f_o are stored at the address of t+4f_o in the output buffer (buf-2). This FC layer performs 512 × 512 MAC operations for a time frame (t). They are divided into f_i × f_o (32 × 32) single-weight-set operations (Equation (6)); a single-weight-set operation executes n_i × n_o (16 × 16) MAC operations with the 16 × 16 systolic array as in the convolution operation (Equation (7), Figure 7). Four single-weight-set operations (t = 0,1,2,3) are repeated f_i × f_o (32 × 32) times; for each f_o, 32 single-weight-set operations are performed and accumulated at the address of t+4f_o in the output buffer (Equation (8)). A set of 16 × 16 weights is fetched from DRAM to FIFO in 108 T for each set of f_i and f_o and is loaded into the systolic array in 21 T. The longest delay (108 T) dominates the delay of the pipeline stage, so it is multiplied by 32 × 32 and the other delay components are added only once in the computation time of the FC operation (Table 9). Thus, the FC operation takes 110,710 T (= 108 × 32 × 32 + 21 + 4 + 4 + 82 + 4 + 3), which corresponds to throughput of 1.1 GMAC/s at a system clock of 120 MHz. The output buffer (buf-2) is used as the input buffer of the subsequent operation.

$$OUT[t, f_o, n_o,] = BIAS[f_o, n_o] + \sum_{f_i=0}^{31} \sum_{n_i=0}^{15} IN[t, f_i, n_i] \times W[n_i, f_i, f_o, n_o] \quad (6)$$

$$PSUM_{FC}[t, f_i, f_o, n_o] = \sum_{n_i=0}^{15} IN[t, f_i, n_i] \times W[n_i, f_i, f_o, n_o] \quad (7)$$

$$OUT[t, f_o, n_o] = BIAS[f_o, n_o] + \sum_{f_i=0}^{31} PSUM_{FC}[t, f_i, f_o, n_o], (f_o = 0, 1, 2, ..., 31) \quad (8)$$

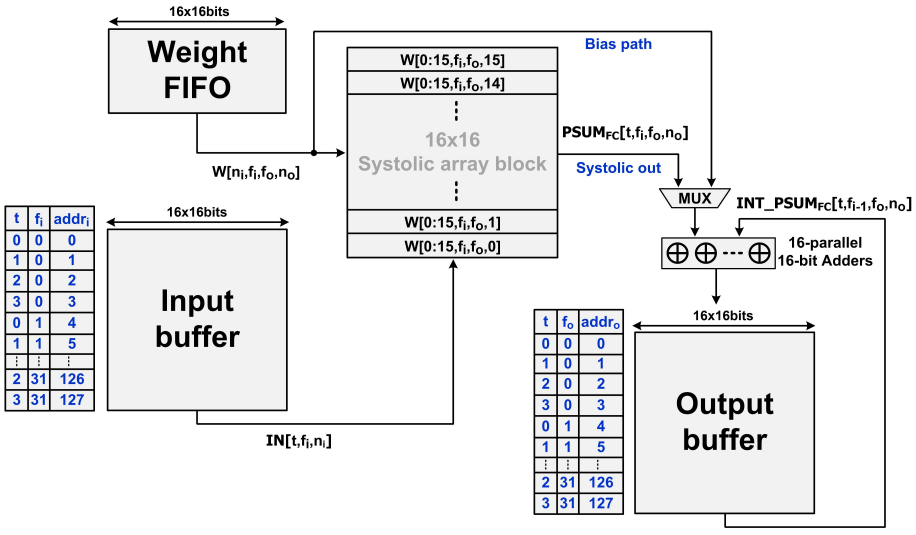


Figure 7. Hardware operation of FC example in Table 8.

Table 9. Clock cycles for each pipeline stage in an example FC operation of Figure 2.

Stage	DRAM Fetch	Weight Load	Input Fetch	Input Cycles	Systolic Array MAC	16-Adders Sum	Result Store
Cycle	108	21	4	4	82	4	3

5. Experimental Results

The low-latency streaming on-device ASR system (Figure 1) was implemented on an ASR system board; a Xilinx Virtex-6 (XC6VLX365T) FPGA chip, an 8 Gb DDR3 DRAM chip, and a USB 2.0 PHY chip are placed on the ASR system board (Figure 8). An Android smartphone with 12 GB DRAM was connected to the ASR system board through the USB 2.0 PHY chip.

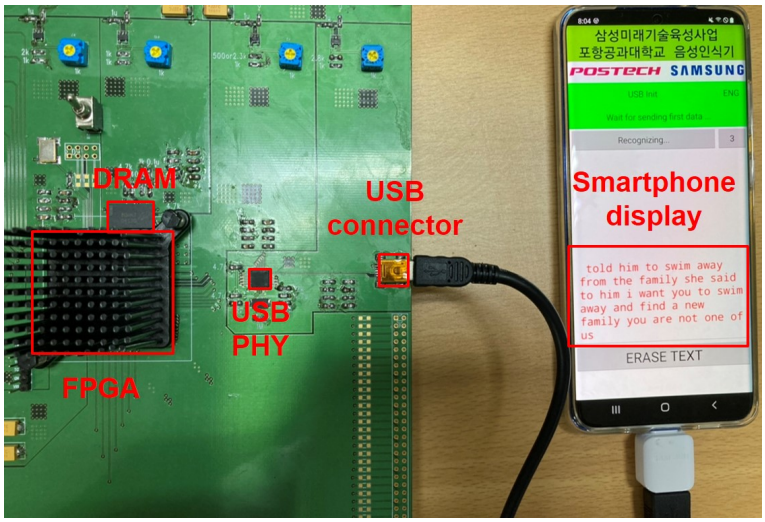


Figure 8. The ASR system board of this work.

When it starts running in the smartphone, the ASR driver program performs an initialization step and repeats a normal operation step. For the initialization step, the ASR driver program loads the three-gram LM dictionary (1.3 GB) from flash to DRAM inside the smartphone, downloads the instruction code (4 kB) and the trained weights (18.6 MB) to the DRAM chip of the ASR system board through USB end-point EP1, and sends a control code (INSTR_FETCH) through EP3 to the AM manager to fetch the 4 kB instruction code from DRAM to the instruction memory block on the ASR system board. For the normal operation step, the ASR driver sprogram downloads the Mel-spectrogram data to the DRAM chip of the ASR system board through EP1, sends a control signal (START_RUN) to the AM manager through EP3 to run the instruction code that is stored in the instruction memory block, runs the LM code that performs beam search on the word-piece probability after receiving a DONE signal through EP4 from the AM manager, then displays text output of the LM. A beam-search decoder with a beam size of 50 was implemented using the KenLM library.

To measure WERs, the audio files of LibriSpeech test-clean were stored in the flash memory of the smartphone. Then, the ASR driver program stores both the word-piece probability calculated by the hardware ASR system and the text output calculated by the ASR driver program of the smartphone in the flash memory, and computes WER by comparing them with the oracle data. WER was measured at 80 ms and 160 ms speech input. The WER of the word-piece probability (AM WER) is 13.2% for 80 ms input and 12.9% for 160 ms input; they are larger by 1.7% and 1.5%, respectively, than the software WER using the IEEE FP16 numbers. This degradation is considered to be due to the simple FP16 number system used in this work. The subnormal number that increases the dynamic range by 10^3 in the IEEE FP16 numbers is not used here; this omission reduced the number

of LUTs in the FPGA chip by half. The WER of text output using the three-gram LM (3 g LM WER) was 9.1% for 80 ms input and 8.8% for 160 ms input; they are larger by 1.2% and 1%, respectively, than the software WER that uses the IEEE FP16 numbers (Table 10).

Table 10. Word error rate of proposed ASR system (#weights: 9.3 M).

LibriSpeech Test-Clean	80 ms Input		160 ms Input	
	SW 16 bit	HW 16 bit	SW 16 bit	HW 16 bit
AM WER (%)	11.5	13.2	11.4	12.9
3-gram LM WER (%)	7.9	9.1	7.8	8.8

To evaluate the speed of the hardware AM, the processing times of all convolutions (CONV2D) and fully connected (FC) operations were measured by monitoring the state change at the master controller in Figure 4 (Table 11). Because the systolic array and the DRAM fetch cycle work in pipeline and the DRAM fetch cycle takes longer than the systolic array, the DRAM fetch cycle determines the speed of the hardware AM. The FC operation takes much longer than the CONV2D operation in the computation time of the systolic array. The DRAM fetch time of the FC weights dominates the processing time with a DRAM bandwidth of 5 Gbps and a system clock frequency of 120 MHz. Compared to the 80 ms unit input, the computational complexity is doubled in the 160 ms unit input, but the total processing time is almost the same due to the weight reuse in the systolic array. The total processing time of the AM was ~34 ms for unit input of 80 ms, and 34.3 ms for a unit input of 160 ms, including layer normalization, residual network, and ReLU operations.

Table 11. Measured performance of CONV2D and FC operations in proposed hardware AM.

Unit Input	80 ms		160 ms	
Operation (#operations)	CONV2D (20)	FC (35)	CONV2D (20)	FC (35)
#weights	51 K	9244 K	51 K	9244 K
#MACs	2.4 M	14 M	4.8 M	28 M
Total processing time (ms)	0.22	33.3	0.26	33.3
-Systolic array active time (ms)	0.21	25.7	0.25	26.1
-DRAM fetch time of the weights (ms)	0.18	32.9	0.18	32.9

The processing time of a CONV2D and FC operation is affected by t, f, k, and A (Table 12). A is the fetch time of 16 × 16 weights from DRAM to FIFO. A is the longest delay in the pipeline stage in this work; A = 108 T at the DRAM bandwidth of 5 Gbps and the system clock frequency of 120 MHz. If the number of weight reuses (number of input cycles: txf in CONV2D, t in FC) is $\geq$ A, then txf or t becomes the dominant pipeline stage (Case 1 in Table 12). The FC operations that take up most of the AM processing time belong to Case 2 (Table 12), because t $\leq$ 8, f_i = 32, f_o = 32, A = 108 T in this work. The numbers 117 and 114 in Table 12 account for the sum of the weight load into the systolic array (21 T), the input fetch delay (7 T in CONV2D, 4 T in FC), the systolic array latency (82 T), the 16-parallel FP16 adders delay (4 T), and the output buffer delay (3 T).

Table 12. Computation time-hand analysis of the CONV2D and the FC operation (#inputs—16; #outputs—16) in the 16 × 16 systolic array (t—#time frames; f—#frequency bins; k—1D kernel shape of CONV2D; f_i—#input frequency bins; f_o—#output frequency bins).

Case	CONV2D Computation Time (Cycles)	FC Computation Time (Cycles)
1. #weight reuses $\leq$ A	txfxk + A + 117	txf_ixf_o + A + 114
2. #weight reuses > A	Axk + txf + 117	Axf_ixf_o + t + 114

The computation times were calculated for the entire AM (Table 13). The hand analysis is calculated using the information in Table 12. The CONV2D operations of Stages 1 and 2 have a number of weight reuses larger than the DRAM fetch time A (Case 1 in Table 12), and all remaining other operations belong to Case 2 in Table 12.

Table 13. Computation time of proposed AM at 80 ms unit input.

Stage	Operation		#Weight Reuses (Input Cycles)	Measured Computation Time (Cycles)	Hand Analysis Computation Time (Cycles)
1	CONV2D		128	1501	1505
2	TDS (#repetitions: 2)	CONV2D	128	1378	1377
		FC	4	111,818	110,710
		FC	4	111,751	110,710
3	CONV2D		64	1242	1261
4	TDS (#repetitions: 3)	CONV2D	64	1135	1153
		FC	2	111,503	110,708
		FC	2	111,611	110,708
5	CONV2D		32	1450	1445
6	TDS (#repetitions: 12)	CONV2D	32	1348	1337
		FC	1	111,704	110,707
		FC	1	111,547	110,707
7	FC		1	199,351	197,807
-	Total		-	4.02 M (33.5 ms)	3.99 M (33.2 ms)

The worst-case latency of the proposed streaming ASR system was 125.5 ms for 80 ms unit input and 217.6 ms for 160 ms unit input. The itemized processing times of the latency show that the unit input interval (80 ms or 160 ms) dominates the latency, and that the FPGA processing time of the AM is the second dominant factor (Table 14). The real-time factors (processing time divided by the unit input time) are 0.57 for 80 ms unit input and 0.36 for 160 ms unit input.

Table 14. Itemized processing times for 80 ms and 160 ms unit input.

	Operation	Audio Path Delay (Smartphone)	Mel-Spectrogram Generation (Smartphone)	Acoustic Model (FPGA)	Beam Search (Smartphone)
80 ms input	Processing time (ms) (avg/max)	4	0.53/0.64	34/35	2.77/5.25
	Total processing time (ms) (avg/max)	41.9/45.5 (including USB transfer of 0.6 ms)			
160 ms input	Processing time (ms) (avg/max)	4	1.05/2.34	34.3/35.3	5.74/14.75
	Total processing time (ms) (avg/max)	46.3/57.6 (including USB transfer of 1.2 ms)			

The computation time of the hardware AM was compared with the software AM (Figure 9). The software AM was implemented by porting the same AM used in the hardware AM to an Android smartphone using Tensorflow Lite. For an 80 ms unit input, the average processing time was 34 ms with the hardware AM and 31 ms with the software AM. For an 160 ms unit input, the average processing time was 34.3 ms with the hardware AM and software AM. With the increase in the unit input length from 80 ms to 160 ms, the AM processing time was increased by only ~1% in the hardware AM because of the weight reuse mostly in the FC layers, whereas it increased by ~48% in the software AM running in

the smartphone. If the DRAM bandwidth was increased to 5.8 Gbps, the processing time of the hardware AM would be reduced to 30.4 ms at the system clock frequency of 120 MHz.

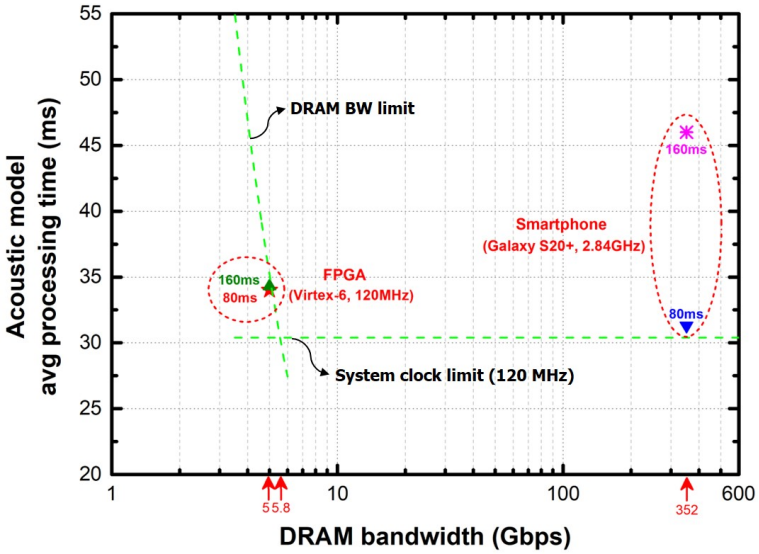


Figure 9. Comparison of processing time of a unit input between the hardware and software AMs.

This system was compared with the prior hardware AMs [16,21] using the LibriSpeech test-clean dataset (Table 15). Most previous methods used LSTM models, but a deep CNN model was used in this work. The MAC units used per mega weight were reduced to less than one-sixth in this work. This method achieved lower WER than the unidirectional LSTM work [16] and was comparable with the bi-directional (non-streaming) LSTM work [22] (Table 15). An ASR system with speech input and a text output display is demonstrated in this work by combining an FPGA, a DRAM, and a smartphone [22].

Table 15. Comparison of ASR inference hardware for LibriSpeech dataset.

ASR Hardware	[21]	[16]	This Work
Frequency (MHz)	573	80/8	**120**
DRAM interface	PCIe (AMBA)	-	**Direct interface**
Acoustic model, #layers	biLSTM, -	uniLSTM, 3	**CNN, 55**
#weights of acoustic model	3.5 M	384 K	**9.3 M**
#MAC units, #gates(NAND2)	1024, 11 M	65, -	**256, 124 K LUTs (~744K gates)**
#MAC units/#mega weights	296	169	**28**
Frames per second	136	>100	**80 ms-input: 229**
WER (%) (LibriSpeech test-clean)	AM: -, LM: 9.4	AM: -, LM: 11.4	**80 ms-input: AM: 13.2, LM: 9.1**
Implementation	ASIC	ASIC	**ASR demo (FPGA+Smartphone)**

6. Discussion

This work achieved a streaming ASR system that has a latency of 125.5 ms by implementing a hardware AM in an FPGA and running a software LM in a smartphone. This work presents a preliminary step to implement a stand-alone ASR system that uses an ASIC chip and a DRAM chip for home-appliance electronic systems that are not connected to a cloud computer. To reduce the hardware size for AM, the input frequency bins were reduced from 80 to 32, the number of the output word-pieces was reduced from 10,000 to 648, and the number of weights for the AM was reduced from 115 M to 9.3 M compared to the software base model [18]. The number of deep-learning layers was increased from 47 to 55 compared to [18] to minimize the WER degradation. To reduce the latency, the

unit input length was set to 80 ms and a direct DRAM interface rather than the AXI bus was used in this work. A USB LINK was implemented in FPGA to communicate with a smartphone to achieve an end-to-end ASR system as a prototype. A programmable architecture was employed in this work to run various CNN-based AMs in FPGA. One of the limitations of this work is the large dictionary size (1.3 GB for the 3-gram LM); it prohibits implementation of a compact hardware ASR system. The future scope of this study could be to implement an AM-only ASR system with decreased WER and reduced latency in an ASIC chip. For example, an AM with more CNN layers than FC layers would help reduce the latency because of the more weight reuse of the CNN layers. The increased bandwidth of DRAM interface would also help reduce the latency.

7. Conclusions

A low-latency on-device streaming ASR system that uses a CNN AM is implemented with FPGA, DRAM, and a smartphone. The proposed hardware AM, which is a modification of a TDS CNN base model, runs on FPGA, and a 3 g software LM runs on the smartphone. To reduce latency and computational complexity, the unit input data of the AM are reduced to 80 ms or 160 ms Mel-spectrogram data with 32 frequency bins, the unit output data are reduced to a probability set of 648 word-pieces, and the number of deep-learning weights is reduced to 9.3 M; the base model has unit input data of 1000 ms, the unit output of a probability set of 10,000 word-pieces, and 115 M weights. The 9.3 M weights in 32-bit floating-point numbers were trained on the 960 h LibriSpeech corpus then converted to 16-bit floating-point numbers without subnormal numbers. On the 80 ms unit input, the AM on FPGA gave WER of 13.2%, and the LM on smartphone gave WER of 9.1% on the LibriSpeech test-clean dataset. Compared to a published streaming LSTM hardware AM, the LM WER was reduced by 2.3%. The demo board using the proposed ASR system works reasonably well. The system latency is 125.5 ms in the worst case; this includes unit input interval of 80 ms, AM processing time of 35 ms on FPGA, LM processing time of 5.25 ms on smartphone, and smartphone audio path delay of 4 ms. The real-time factor of the system is 0.57.

Author Contributions: Conceptualization, J.P. and H.-J.P.; hardware design, J.P. and W.-C.L.; software, H.N. (Hyeonkyu Noh); smartphone program, H.N. (Hyunjoon Nam); project administration, H.-J.P.; J.P. and H.-J.P. worked together during the whole editorial process of the manuscript. All authors were involved in the preparation of this manuscript. All authors have read and agreed to the published version of the manuscript

Funding: This work was supported in part by the Samsung Research Funding and Incubation Center for Future Technology under Grant SRFC-TB1703-04; in part by Samsung Electronics Co., Ltd. (IO201209-07912-01); in part by Basic Science Research Program through the National Research Foundation of Korea funded by the Ministry of Education (2019R1A5A1027055); and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (2022R1A2C2003451).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Alom, M.; Tha, T.; Yakopcic, C.; Westberg, S.; Sidike, P.; Nasrin, M.; Hasan, M.; Essen, B.; Awwal, A.; Asari, V. A State-of-the-Art Survey on Deep Learning Theory and Architectures. *Electronics* **2019**, *8*, 292. [CrossRef]
2. Hsu, W.N.; Bolte, B.; Tsai, Y.-H.H.; Lakhotia, K.; Salakhutdinov, R.; Mohamed, A. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Trans. Audio Speech Lang. Process.* **2021**, *29*, 3451–3460. [CrossRef]
3. Park, D.S.; Chan, W.; Zhang, Y.; Chiu, C.; Zoph, B.; Cubuk, E.D.; Le, Q.V. Specaugment: A simple data augmentation method for automatic speech recognition. In Proceedings of the Interspeech 2019, Graz, Austria, 15–19 September 2019; pp. 2613–2617. [CrossRef]

4. Bahdanau, D.; Chorowski, J.; Serdyuk, D.; Brakel, P.; Bengio, Y. End-to-end attention-based large vocabulary speech recognition. In Proceedings of the 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Shanghai, China, 20–25 March 2016; pp. 4945–4949. [CrossRef]
5. Chan, W.; Jaitly, N.; Le, Q.V.; Vinyals, O. Listen, Attend and Spell: A Neural Network for Large Vocabulary Conversational Speech Recognition. In Proceedings of the 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Shanghai, China, 20–25 March 2016; pp. 4960–4964. [CrossRef]
6. Amodei, D.; Ananthanarayanan, S.; Anubhai, R.; Bai, J.; Battenberg, E.; Case, C.; Casper, J.; Catanzaro, B.; Cheng, Q.; Chen, G.; et al. Deep speech 2: End-to-end Speech Recognition in English and Mandarin. *arXiv* **2015**, arXiv:1512.02595.
7. Jo, J.; Kung, J.; Lee, Y. Approximate LSTM Computing for Energy-Efficient Speech Recognition. *Electronics* **2020**, *9*, 2004. [CrossRef]
8. Beno, L.; Pribis, R.; Drahos, P. Edge Container for Speech Recognition. *Electronics* **2021**, *10*, 2420. [CrossRef]
9. Shi, Y.; Wang, Y.; Wu, C.; Yeh, C.-F.; Chan, J.; Zhang, F.; Le, D.; Seltzer, M. Emformer: Efficient memory transformer based acoustic model for low latency streaming speech recognition. In Proceedings of the 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Toronto, ON, Canada, 6–11 June 2021; pp. 6783–6787. [CrossRef]
10. Chen, X.; Wu, Y.; Wang, Z.; Liu, S.; Li, J. Developing real-time streaming transformer transducer for speech recognition on large-scale dataset. In Proceedings of the 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Toronto, ON, Canada, 6–11 June 2021; pp. 5904–5908. [CrossRef]
11. He, Y.; Sainath, T.N.; Prabhavalkar, R.; McGraw, I.; Alvarez, R.; Zhao, D.; Rybach, D.; Kannan, A.; Wu, Y.; Pang, R.; et al. Streaming End-to-end Speech Recognition For Mobile Devices. In Proceedings of the 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brighton, UK, 12–17 May 2019; pp. 6381–6385. [CrossRef]
12. Kim, K.; Lee, K.; Gowda, D.; Park, J.; Kim, S.; Jin, S.; Lee, Y.-Y.; Yeo, J.; Kim, D.; Jung, S.; et al. Attention based on-device streaming speech recognition with large speech corpus. In Proceedings of the 2019 IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU), Singaopre, 14–18 December 2019; pp. 956–963. [CrossRef]
13. Han, S.; Kang, J.; Mao, H.; Hu, Y.; Li, X.; Li, Y.; Xie, D.; Luo, H.; Yao, S.; Wang, Y.; et al. ESE: Efficient speech recognition engine with sparse LSTM on FPGA. In Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), Monterey, CA, USA, 22–24 February 2017; pp. 75–84. [CrossRef]
14. Wang, S.; Li, Z.; Ding, C.; Yuan, B.; Qiu, Q.; Wang, Y.; Liang, Y. C-LSTM: Enabling efficient LSTM using structured compression techniques on FPGAs. In Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), Monterey, CA, USA, 25–27 February 2018; pp. 11–20. [CrossRef]
15. Cao, S.; Zhang, C.; Yao, Z.; Xiao, W.; Nie, L.; Zhan, D.; Liu, Y.; Wu, M.; Zhang, L. Efficient and Effective Sparse LSTM on FPGA with Bank-Balanced Sparsity. In Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), Seaside, CA, USA, 24–26 February 2019; pp. 63–72. [CrossRef]
16. Kadetotad, D.; Yin, S.; Berisha, V.; Chakrabarti, C.; Seo, J. An 8.93 TOPS/W LSTM recurrent neural network accelerator featuring hierarchical coarse-grain sparsity for on-device speech recognition. *IEEE J. Solid-State Circuits* **2020**, *55*, 1877–1887. [CrossRef]
17. Zheng, S.; Ouyang, P.; Song, D.; Li, X.; Liu, L.; Wei, S.; Yin, S. An Ultra-Low Power Binarized Convolutional Neural Network-Based Speech Recognition Processor with On-Chip Self-Learning. *IEEE Trans. Circuits Syst.-I* **2019**, *66*, 4648–4661. [CrossRef]
18. Pratap, V.; Xu, Q.; Kahn, J.; Avidov, G.; Likhomanenko, T.; Hannun, A.; Lipchinsky, V.; Synnaeve, G.; Collobert, R. Scaling up online speech recognition using convnets. In Proceedings of the Interspeech, Shanghai, China, 25–29 October 2020; pp. 3376–3380. [CrossRef]
19. Greenberg, S.; Carvey, H.; Hitchcock, L.; Chang, S. Temporal properties of spontaneous speech—A syllable-centric perspective. *J. Phon.* **2003**, *31*, 465–485. [CrossRef]
20. Panayotov, V.; Chen, G.; Povey, D.; Khudanpur, S. Librispeech: An asr corpus based on public domain audio books. In Proceedings of the 2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brisbane, Australia, 19–24 April 2015; pp. 5206–5210. [CrossRef]
21. Tambe, T.; Yang, E.; Ko, G.G.; Chai, Y.; Hooper, C.; Donato, M.; Whatmough, P.N.; Rush, A.M.; Brooks, D.; Wei, G. A 25 mm^2 Soc for IoT Devices with 18 ms Noise-Robust Speech-to-Text Latency via Bayesian Speech Denoising and Attention-Based Sequence-to-Sequence DNN Speech Recognition in 16 nm FinFET. In Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers, San Francisco, CA, USA, 20–24 February 2021; pp. 158–159. [CrossRef]
22. Demo of Automatic Speech Recognition System, POSTECH, CNN-Based Streaming ASR in FPGA and Smartphone. Available online: https://www.youtube.com/watch?v=-LB9H-uM-zU (accessed on 12 August 2021).